

Comparing KNN, SVM, MLP and CNN Classifiers on Handwritten Digit Recognition

Tomás Ferreira (2025168427)

University of Coimbra, Portugal

Abstract. This report compares k-nearest neighbours (KNN), support vector machines (SVM) and multi-layer perceptrons (MLP) on the scikit-learn Digits dataset and, as extra work, on MNIST, where a convolutional neural network (CNN) is added. All models follow the same protocol: a stratified 60/20/20 train/validation/test split, hyperparameter selection by validation macro F1, retraining of the selected configuration on training and validation data, and a single evaluation on the test set. On Digits the three classifiers reach 98.1–98.6% test accuracy and differ by only one or two misclassified images. On MNIST, KNN prediction time grows almost linearly with the number of stored training images, which explains its cost on large datasets; SVM and MLP reach 98.3–98.4%, and the CNN obtains the best result (98.85% accuracy, macro F1 0.9884).

Keywords: Classification · KNN · SVM · MLP · CNN · MNIST

1 Introduction

The assignment asks to construct classifiers of different types, train them and compare their performance with scikit-learn. After a short warm-up on the Iris dataset (test accuracy 0.92), the main study uses the Digits dataset with three classifiers: KNN, SVM and MLP. As extra work, the study is repeated on MNIST, where KNN is expected to become slow, and a convolutional classifier is added using PyTorch. All experiments are implemented in a single Jupyter notebook. During development, trained models were stored as checkpoints to avoid unnecessary retraining and to support reproducible evaluation.

2 Methodology

2.1 Data Split and Model Selection

Each dataset is split once, with stratification and a fixed random seed, into training (60%), validation (20%) and test (20%) sets, and all models use exactly the same subsets (Fig. 1). Candidate configurations are fitted on the training set and compared on the validation set; the selected configuration is then retrained

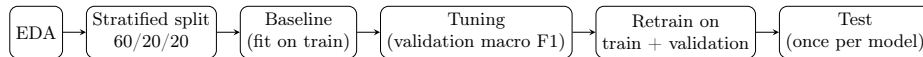


Fig. 1. Experimental protocol applied to every classifier. The test set is used only once, by the final models.

from scratch on training plus validation data and evaluated once on the test set, which is never used for any decision.

Macro F1 was chosen as the primary selection metric because both Digits and MNIST are multiclass problems with 10 classes and relatively balanced class distributions (174–183 images per class in Digits, 6,313–7,877 in MNIST). Macro F1 averages the per-class F1 scores, so every digit contributes equally to model selection and weak performance on one class is not hidden by the others. Accuracy, macro and weighted precision/recall/F1 and confusion matrices are reported as complementary measures.

2.2 Preprocessing

Digits pixels are integers in 0–16. For SVM and MLP a `StandardScaler` is placed inside a scikit-learn pipeline, so its statistics are learned only from the rows used for fitting (no leakage). Several border pixels are almost constant, and standardisation turns their rare non-zero values into extreme inputs ($|z|$ up to 33 in training); for KNN, scaling was therefore treated as a hyperparameter. MNIST pixels (0–255) are divided by 255 for all models, a fixed transform; standardisation was not used, since 76 training pixels have zero variance and others would reach $|z| \approx 205$.

3 Digits Dataset

Digits contains 1,797 grayscale images of 8×8 pixels (64 features), with no missing values or duplicates. The exploratory analysis showed that the class-mean images of digits 1 and 8 are the most similar (correlation 0.87), anticipating confusions involving digit 8.

On the validation set, 72 KNN configurations (k , uniform/distance weights, with/without scaling), 27 SVM configurations (linear, RBF and polynomial kernels with several values of C and γ) and 14 MLP configurations (hidden layers, activation, α and learning rate, varied one at a time) were compared. Four MLP configurations raised convergence warnings and were rerun with more iterations. Standardisation reduced KNN validation macro F1 for every k tested (best 0.9751 vs. 0.9889 without scaling); a change in the relative weight of low-variance peripheral pixels in the distance may contribute, but the experiment does not isolate the cause.

On the test set (Table 1) the three classifiers reach 98.1–98.6% accuracy. KNN obtained the highest accuracy and macro F1 on this split (5 errors), followed by SVM (6) and MLP (7). These differences correspond to one or two images, so

they do not show a substantial difference between the models. Accuracy and macro F1 are almost identical, as expected for balanced classes, and confusions involving digit 8 were frequent ($8 \rightarrow 1$ accounts for 3 errors of both KNN and MLP).

4 Extra Work: MNIST and CNN

MNIST has 70,000 images of 28×28 pixels (784 features). The official training and test parts were pooled and re-split with the same protocol (42,000/14,000/14,000 images), so the results are not directly comparable with those reported on the official test split.

4.1 KNN Scalability

KNN is a lazy learner: fitting only stores the training set, while each prediction computes the distance to every stored image, a cost approximately proportional to $n_{\text{train}} \times n_{\text{features}}$. MNIST increases both factors ($39\times$ more samples and $12\times$ more features than Digits). KNN ($k = 5$) was fitted on stratified training subsets of 1,000–42,000 images and timed on the same 2,000 validation images (Fig. 2a). Prediction time grew from 0.055 s to 1.03 s, close to linearly ($R^2 = 0.998$), while fitting stayed below 0.1 s; per query, MNIST was $21\times$ more expensive than Digits. With vectorised distance computations the full KNN remained feasible on this hardware (about 8 s for the 14,000 test images), but it was the least accurate model.

4.2 SVM and MLP

SVC training time grew approximately as $n^{2.07}$ (measured on 2,500–10,000 images). SVM hyperparameters (RBF kernel, $C \in \{1, 10\}$, $\gamma \in \{\text{scale}, 0.05\}$) were therefore compared on a stratified 10,000-image training subset, and only the selected model was trained on the full 56,000 images. The MLP was tuned on the full training set (architecture, then α); the selected (256, 128) network converged without warnings.

4.3 Convolutional Neural Network

The CNN has two convolutional blocks, Conv(32, 3×3)–ReLU–MaxPool and Conv(64, 3×3)–ReLU–MaxPool, followed by a 128-unit dense layer and a 10-class output (421,642 parameters). It was trained with Adam (learning rate 10^{-3}), batch size 128 and cross-entropy loss, with early stopping on validation macro F1 (patience 3). The best epoch was 8 (validation macro F1 0.9885), after which the validation loss stopped improving while the training loss kept decreasing (Fig. 2b). The final CNN was retrained for 8 epochs on training plus validation data.

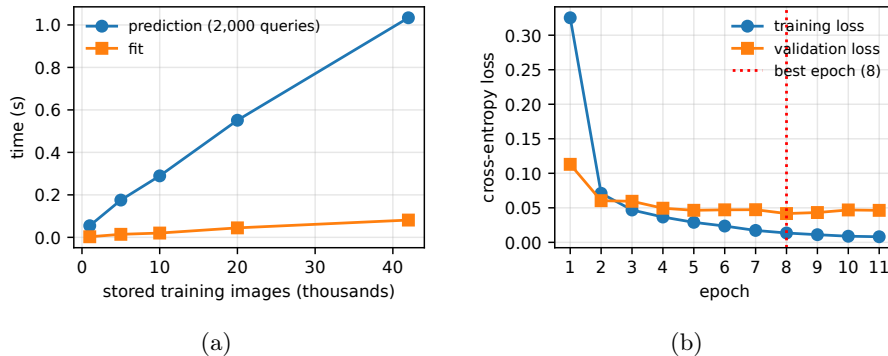


Fig. 2. MNIST. (a) KNN fit and prediction time vs. number of stored training images (2,000 validation queries). (b) CNN training and validation loss; early stopping selects epoch 8.

Table 1. Selected configurations and results. Validation macro F1 was used for selection; test metrics come from the final models, trained on training plus validation data.

	Model (selected configuration)	Val. F1	Test acc.	Test F1	Errors
Digits	KNN ($k=2$, distance, no scaling)	0.9889	0.9861	0.9859	5/360
	SVM (RBF, $C=100$, $\gamma=0.01$)	0.9944	0.9833	0.9832	6/360
	MLP (100, ReLU, lr=0.01)	0.9861	0.9806	0.9802	7/360
MNIST	KNN ($k=3$, distance)	0.9713	0.9732	0.9731	375/14k
	SVM (RBF, $C=10$, $\gamma=\text{scale}$)*	0.9672	0.9834	0.9833	232/14k
	MLP (256-128, $\alpha=10^{-3}$)	0.9803	0.9843	0.9841	220/14k
	CNN (2 conv blocks, 8 epochs)	0.9885	0.9885	0.9884	161/14k

* tuned on a 10,000-image training subset.

5 Results

On MNIST the CNN obtained the best test performance (161 errors), followed by MLP (220), SVM (232) and KNN (375); MLP and SVM obtained very similar results, and $4 \rightarrow 9$ was the most frequent confusion for all four models. Computational costs differ strongly: SVM was the most expensive model to train (152s) and to predict (74s, with 10,972 support vectors), KNN pays its cost at prediction time, and the MLP was the fastest at prediction (0.06s); the CNN was trained on a GPU in 16s, so its times are not directly comparable with the CPU-based models. The CNN preserves the 2D structure of the images, a plausible advantage over the flattened MLP input; however, the two models also differ in parameter count and training procedure, so the gain cannot be attributed to convolution alone.

6 Conclusions

On Digits, KNN, SVM and MLP achieved very similar high performance on this split. On MNIST, the larger scale exposes computational differences: KNN inference cost grows with the size of the training set, SVM training became substantially more expensive as the number of samples increased, and the CNN obtained the best result.